Who Wrote It? Identifying LLM Families from Their Responses

CMPSC 448 Midterm Project, Fall 2026

Team leader: [LEADER NAME]
Team members: [MEMBER NAME 1], [MEMBER NAME 2], [MEMBER NAME 3]

October 11, 2026

Repository: `[GITHUB URL]`. All numbers in this report are generated by `src/make_report.py` from the JSON files in `results/`; nothing is typed by hand.

1. Problem definition and dataset curation

Task. Given a response written by a large language model, predict which *LLM family* produced it. This is a 5-way text classification problem. We also ask whether the user prompt helps (RQ2), whether the learned "fingerprint" transfers across task types (RQ3), and which stylistic signals separate the families (RQ4).

Source. We use the public **UltraFeedback** dataset (`openbmb/UltraFeedback`, MIT licence; Cui et al., 2023). Each record has a prompt (`instruction`), the sub-dataset it came from (`source`: Evol-Instruct, ShareGPT, UltraChat, FLAN, TruthfulQA, FalseQA), and four completions, each tagged with the generating `model`. This gives the required (LLM_name, LLM_input, LLM_output) triples directly. The raw files contain 255,864 completions for 63,936 unique prompts from 17 models. Because every prompt was answered by four randomly chosen models, **the same prompt appears with several different labels**, which matters for RQ2. No private data is involved; we did not query any model ourselves.

Family grouping. We grouped models by developer and kept only unambiguous ones:

family	models
GPT (OpenAI)	gpt-3.5-turbo, gpt-4
LLaMA-2 (Meta)	llama-2-7b-chat, llama-2-13b-chat, llama-2-70b-chat
Bard (Google, predecessor of Gemini)	bard
Falcon (TII)	falcon-40b-instruct
MPT (MosaicML)	mpt-30b-chat

We dropped third-party LLaMA fine-tunes (Vicuna, WizardLM, UltraLM, Alpaca): they share LLaMA's base weights but were tuned largely on GPT-distilled data, so assigning them to either family would blur the labels. We also dropped StarChat (a code-only model) and Pythia-12B (only 834 completions).

Cleaning and balancing. Starting from 124,531 completions of the kept models, we removed empty texts, exact duplicate responses and duplicate (prompt, model) pairs, and kept responses of 10–1000 words. 116,029 rows remained, but they were very unbalanced (LLaMA-2 49,702 vs Bard 8,266). We randomly downsampled every family to 4,000 rows, giving **20,000 examples** and 17,899 unique prompts.

Task label. For RQ3 we derived a task label for every prompt. TruthfulQA and FalseQA prompts become *factual_qa*. For all others we apply keyword rules on the prompt, in order: code terms give *coding*; math terms or arithmetic give *math*; remaining FLAN prompts give *nlp_task*; writing verbs (write, essay, story, email, …) give *writing*; everything else is *chat*.

task	rows	Bard	Falcon	GPT	LLaMA-2	MPT
writing	2336	576	415	499	394	452

task	rows	Bard	Falcon	GPT	LLaMA-2	MPT
coding	3419	690	682	706	641	700
chat	7666	1851	1381	1634	1350	1450
math	1686	257	374	331	373	351
nlp_task	3731	258	974	622	1041	836
factual_qa	1162	368	174	208	201	211

Split. We split 80/10/10 *by unique prompt* (seed 448), so a prompt and all its responses fall in a single split. The code asserts that no prompt crosses splits. After downsampling, 1,880 prompts are still answered by more than one family.

split	total	Bard	Falcon	GPT	LLaMA-2	MPT
train	16009	3224	3180	3176	3207	3222
val	1999	405	426	377	405	386
test	1992	371	394	447	388	392

2. Models and training

Tokenisation. We use a case-preserving regex tokenizer (`\n|\w+|[^\w\s]`). Every punctuation mark is its own token and newlines become a `<nl>` token, so formatting stays visible to the models. The vocabulary is built from the training split only (minimum frequency 2, capped at 30,003 entries, including `<pad>`, `<unk>` and `<sep>`). Sequences are truncated to 256 tokens. Batches are padded dynamically, and padding is masked out of every pooling operation.

CNN. Embedding (128-d), then three parallel Conv1d layers with kernel sizes 3, 4 and 5 (128 filters each, ReLU), then masked max-over-time pooling, concatenation (384-d), dropout 0.5 and a linear layer to 5 classes. 4,039,301 parameters.

RNN (BiLSTM). Embedding (128-d), a 1-layer bidirectional LSTM (64 hidden units per direction), then masked mean pooling and masked max pooling over time, concatenated (256-d), dropout 0.5 and a linear layer to 5 classes. 3,940,997 parameters.

Training. Adam (learning rate 1e-3), cross-entropy loss, batch size 64, gradient clipping at 5. Batches are bucketed by length to reduce padding. We train for at most 6 epochs with early stopping on validation accuracy (patience 2) and restore the best-validation weights before testing. Every configuration is run with seeds 1, 2 and 3, and we report mean ± standard deviation of test accuracy and macro-F1. For RQ1/RQ2 the best validation epochs were 5, 6, 4 (CNN) and 6, 5, 5 (BiLSTM).

Hardware and run time. All RQ1/RQ2 runs were trained on a laptop CPU (16 logical cores, PyTorch 2.9.1 CPU build), four runs in parallel with four threads each. One output-only run took 7.8 min (CNN) or 19.7 min (BiLSTM) on average. Our first BiLSTM (128 hidden units, packed sequences) needed more than 10 minutes per epoch. We therefore halved the hidden size and dropped sequence packing, which was about 3× slower on this CPU. Masked pooling keeps padding out of the representation, so packing is not needed for correctness. Every result file records the device it ran on.

Baselines. (a) TF-IDF over word unigrams and bigrams (case-preserving, punctuation kept, sublinear tf), followed by logistic regression (C = 4). (b) A gradient-boosting classifier on 34 hand-crafted style features (RQ4).

3. Results

RQ1: Can we identify which LLM generated a response?

Output-only classification on the held-out test split (1,992 responses, about 400 per family):

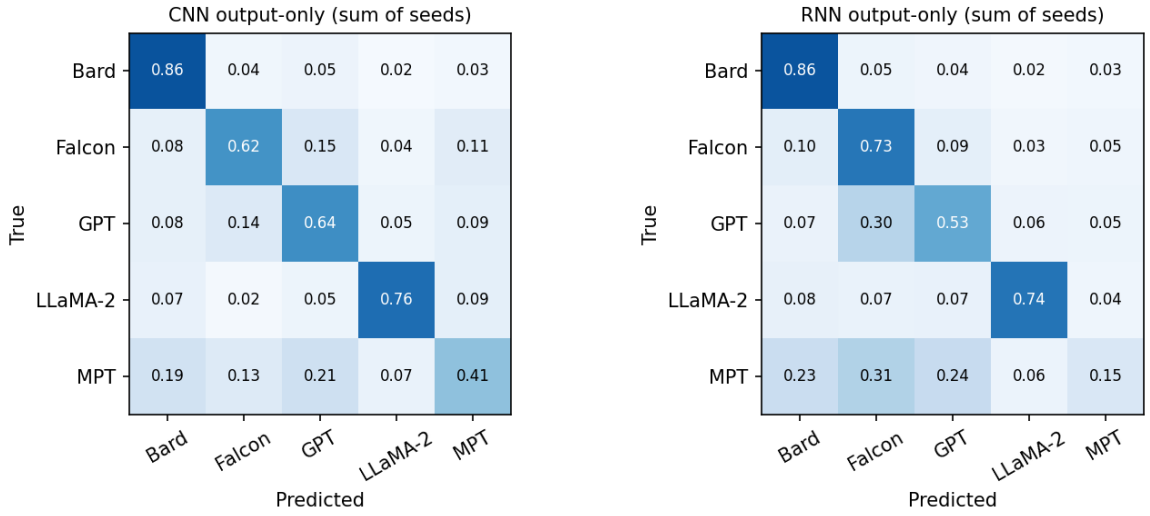
model	test accuracy (%)	macro-F1 (%)	seeds
CNN (ours)	65.6 ± 0.5	65.0 ± 0.6	3
BiLSTM (ours)	59.6 ± 1.7	57.1 ± 2.4	3
TF-IDF + LogReg (baseline)	73.6	73.9	1 (deterministic)
Hand-crafted features + GBoost (RQ4)	68.2	68.3	1

model	test accuracy (%)	macro-F1 (%)	seeds
Chance (uniform)	20.0	-	-

Both neural models are far above chance (20%). The **CNN (65.6 ± 0.5%) beats the BiLSTM (59.6 ± 1.7%)** by about six points and is also about 2.5× faster to train. The simple TF-IDF baseline is stronger still (73.6%); Section 5 explains why.

Per-family test accuracy (recall), mean over seeds:

family	CNN	BiLSTM	TF-IDF
Bard	86.4 ± 2.8	86.1 ± 1.7	88.1
Falcon	61.9 ± 13.2	72.8 ± 7.3	68.5
GPT	63.8 ± 8.4	52.6 ± 8.3	74.7
LLaMA-2	76.5 ± 2.4	73.7 ± 5.4	79.6
MPT	41.1 ± 11.6	15.1 ± 4.5	57.7



Bard is the easiest family for every model (86.4% with the CNN), followed by **LLaMA-2** (76.5%). **MPT** is the hardest (41.1% with the CNN, 15.1% with the BiLSTM). The CNN sends 20.7% of MPT responses to GPT and 19.0% to Bard. MPT-30B-chat's fine-tuning data includes ShareGPT conversations, which are largely ChatGPT outputs, so a GPT-like style is plausible. Falcon and GPT are also often confused with each other.

RQ2: Does the user's prompt help identify the LLM?

The same code, vocabulary procedure and hyperparameters are used for all three setups. *Input + output* is the prompt (first 64 tokens), then <sep>, then the response, truncated to 256 tokens in total.

setup	CNN acc	BiLSTM acc	TF-IDF acc	CNN F1	BiLSTM F1	TF-IDF F1
input only (prompt)	21.2 ± 0.9	22.5 ± 0.3	22.1	16.1 ± 0.3	18.2 ± 2.3	21.7
output only (response)	65.6 ± 0.5	59.6 ± 1.7	73.6	65.0 ± 0.6	57.1 ± 2.4	73.9
input + output	65.0 ± 0.8	57.5 ± 1.2	70.5	64.9 ± 1.1	55.6 ± 1.4	70.9

(i) Can the LLM be predicted from the prompt alone? Essentially no. Input-only accuracy is 21.2 ± 0.9% (CNN), 22.5 ± 0.3% (BiLSTM) and 22.1% (TF-IDF). That is barely above uniform chance (20%) and the test majority class rate (22.4%). This is

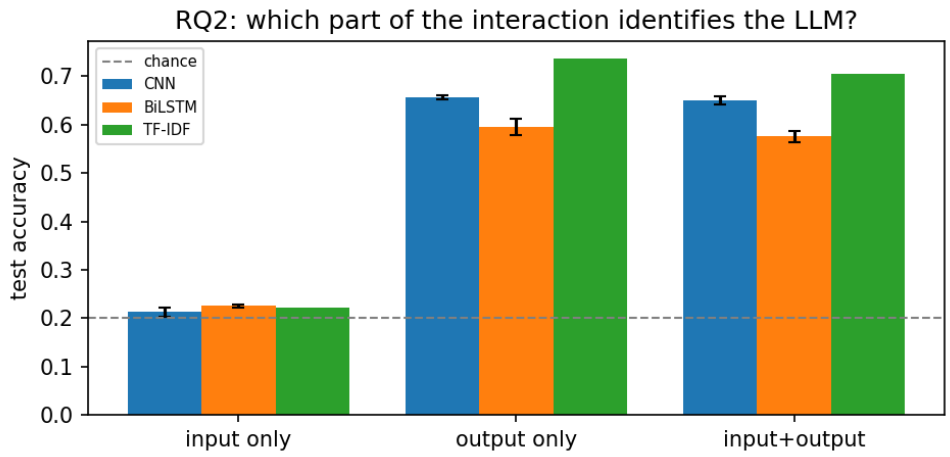


Figure 1: RQ2: test accuracy by input setup (error bars: std over 3 seeds).

what we expected: UltraFeedback assigned models to prompts at random, so the prompt says almost nothing about who answered it. Prompts also repeat across families. 3,865 rows share their prompt with a response from a different family, and identical inputs with different labels cannot all be classified correctly. Even an oracle that always predicted the most common family per test prompt would be capped at 90.9%.

Is the small gain above chance real, and where does it come from? A one-sided binomial test of the TF-IDF input-only accuracy against 20% gives p = 0.010. Against the majority-class rate it gives p = 0.63, so the model does no better than always guessing the largest class. The little signal that exists is **dataset leakage, not a property of the models**. Bard has no responses at all in three FLAN sub-sets (NIv2, P3, FLAN-2021) and is over-represented in Evol-Instruct and UltraChat, so the prompt's *source* is weakly informative. A logistic regression on the one-hot source alone reaches 28.1%, more than any text model. Almost all of that signal identifies Bard: TF-IDF input-only recall for Bard is 39.4%, while the other families are near or below 20%. The derived task label (21.9%) and prompt length (19.5%) carry no signal; median prompt lengths are Bard 36, Falcon 52, GPT 42, LLaMA-2 56, MPT 53 words.

(ii) Does adding the input improve classification? No. It never helps, and it hurts slightly for the BiLSTM and TF-IDF; for the CNN the difference is within seed noise. Input + output is 65.0 ± 0.8% for the CNN vs 65.6 ± 0.5% output-only, 57.5 ± 1.2% vs 59.6 ± 1.7% for the BiLSTM, and 70.5% vs 73.6% for TF-IDF. Prepending the prompt uses up part of the fixed 256-token budget, which cuts off part of the response, and the prompt tokens themselves are mostly noise for this task.

(iii) When could input-only work "surprisingly well"? When the data collection links prompts to models: each model answering a different prompt pool, different system prompts, or sources with different topics. A classifier would then learn the *collection process* rather than the LLM. Our near-chance result, together with the source-only check, suggests our split is largely free of this effect. The remaining effect is small and fully explained by Bard's source distribution.

4. In-depth analysis

RQ3: Do LLM fingerprints generalise across tasks?

Setup. For each task *T* we train the CNN (3 seeds) on the training split *without any task-T rows*, early-stop on validation rows not from *T*, and test on task-*T* rows. We compare against the in-domain CNN from RQ1, whose training data includes task *T*, *on exactly the same test rows*. As a larger second sample we also report accuracy on *all* task-*T* rows across all splits, since the held-out model never saw any of them. For *writing* and *coding* we repeat the experiment with the BiLSTM.

held-out task	model	n test	in-domain acc	held-out acc (same rows)	Δ (pts)	held-out acc, all rows of task
writing	CNN	246	57.7 ± 3.1	55.8 ± 2.9	-1.9	59.6 ± 1.5 (n=2,336)

held-out task	model	n test	in-domain acc	held-out acc (same rows)	Δ (pts)	held-out acc, all rows of task
writing	BiLSTM	246	59.8 ± 3.2	54.7 ± 1.8	-5.0	56.3 ± 1.3 (n=2,336)
coding	CNN	343	70.7 ± 2.2	62.1 ± 3.5	-8.6	61.6 ± 2.3 (n=3,419)
coding	BiLSTM	343	60.0 ± 2.4	55.3 ± 2.1	-4.7	54.7 ± 1.5 (n=3,419)
chat	CNN	757	70.5 ± 0.6	62.6 ± 3.3	-7.9	62.9 ± 3.3 (n=7,666)
math	CNN	175	60.8 ± 3.3	61.5 ± 2.7	+0.8	61.3 ± 1.5 (n=1,686)
nlp_task	CNN	359	59.7 ± 1.5	56.5 ± 2.4	-3.2	57.3 ± 1.5 (n=3,731)
factual_qa	CNN	112	61.0 ± 3.4	55.7 ± 2.7	-5.4	57.0 ± 2.1 (n=1,162)

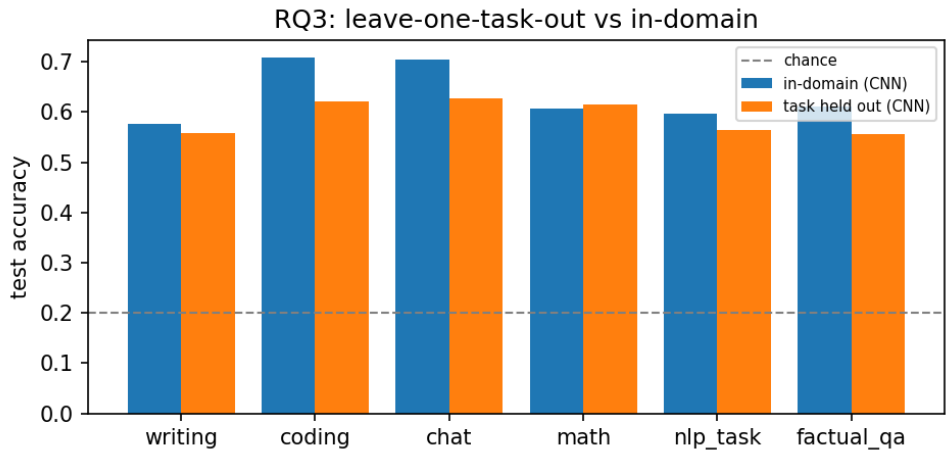


Figure 2: RQ3: CNN accuracy on each task when the task is in the training data vs held out.

Per-family accuracy of the held-out CNN on all rows of the unseen task (mean of 3 seeds, %):

held-out task	Bard	Falcon	GPT	LLaMA-2	MPT
writing	86.2	53.7	55.1	57.2	38.1
coding	54.2	54.3	55.4	82.9	62.7
chat	88.3	56.0	47.0	83.9	35.7
math	73.5	67.9	51.6	74.4	40.7
nlp_task	80.2	64.5	54.6	68.7	29.6
factual_qa	70.4	80.1	10.9	89.4	29.4

(i) Does performance drop on unseen tasks? Yes, but only moderately. Averaged over the six tasks, holding the task out changes CNN accuracy on that task by -4.4 points. The largest drop is on *coding* (-8.6); the smallest change is on *math* (+0.8). Every held-out accuracy (55–63%) stays far above chance (20%). For the two required cases: trained without writing, the CNN reaches 55.8 ± 2.9% on writing prompts vs 57.7 ± 3.1% in-domain (-1.9 pts). Trained without coding, it reaches 62.1 ± 3.5% vs 70.7 ± 2.2% (-8.6 pts). The BiLSTM shows the same pattern: writing 54.7 ± 1.8% vs 59.8 ± 3.2%, coding 55.3 ± 2.1% vs 60.0 ± 2.4%. Two caveats. The per-task test sets are small (112–757 rows), so seed standard deviations of 2–4 points mean the smaller differences are within noise. And held-out models also train on somewhat less data.

(ii) Are some families more consistently identifiable? Yes. On every unseen task, **LLaMA-2** stays at or near the top (held-out recall between 57% on writing and 89% on factual QA). Its signature is task-independent: chatty openings ("Sure, I'd be happy to help", "Hello! I'm here to…") and safety caveats. **Bard** is also robust (86% on unseen writing) *except on coding* (54%), where its usual bullet-and-bold layout gives way to code. **GPT** is the least stable: 11% on unseen factual QA, where its short, plain answers look like Falcon or MPT. **MPT** is weak on most tasks (36% on chat, 29% on factual QA) and only does well on coding (63%).

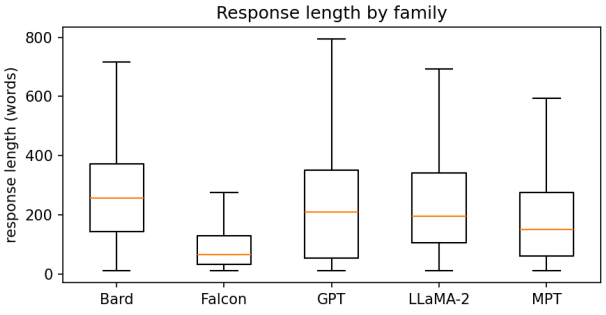
(iii) General fingerprint or task-specific patterns? Mostly general, partly task-specific. Most of the in-domain accuracy survives without any example of the target task, so the models do learn a family style that transfers. The moderate drops, and the family-specific collapses (Bard on coding, GPT on factual QA), show that part of what is learned is "how family X answers task Y". Section 4 (RQ4) points to formatting and length habits; these are strongly task-dependent (code blocks, lists), which explains why they transfer imperfectly.

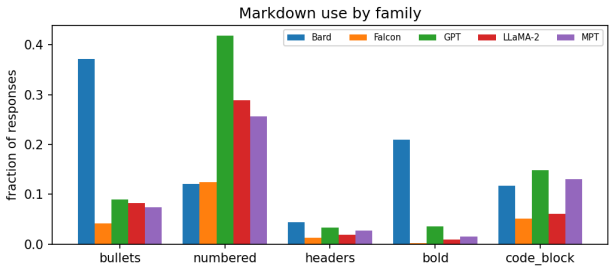
RQ4: What characteristics distinguish the LLMs?

Per-family statistics (all 4,000 responses per family):

statistic	Bard	Falcon	GPT	LLaMA-2	MPT
median length (words)	255	64	209	195	151
type-token ratio, first 100 words	0.62	0.76	0.71	0.68	0.67
mean word length (chars)	4.72	4.90	5.13	4.92	4.94
% with bullet list	37	4	9	8	7
% with numbered list	12	12	42	29	26
% with **bold**	21	0	4	1	1
% with code block	12	5	15	6	13
commas per 100 words	5.1	6.4	8.8	6.0	6.9
'!' per 100 words	0.13	0.37	0.19	0.80	0.21
% opening with Sure/Certainly/Of course	16	6	5	31	11
% opening with Hello/Hi	0	1	0	17	1
% mentioning 'AI language model'	1	10	5	3	0

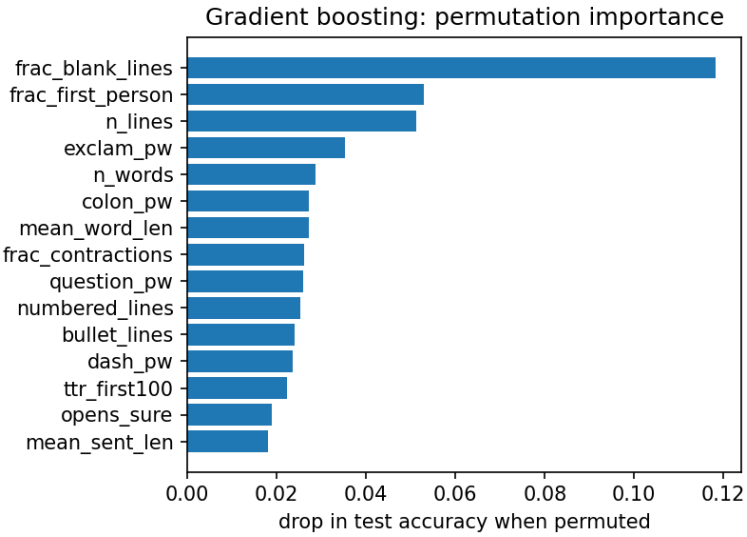
family	most frequent 3-word openings (share of family's responses)
Bard	"sure, here is" (5.1%); "sure, i can" (3.3%); "here is a" (2.5%)
Falcon	"as an ai" (2.7%); "i'm sorry, i" (1.5%); "thank you for" (1.5%)
GPT	"as an ai" (2.2%); "as an ai," (1.1%); "to create a" (1.0%)
LLaMA-2	"sure, i'd be" (12.9%); "thank you for" (9.5%); "hello! i'm here" (8.4%)
MPT	"based on the" (1.5%); "sure, i can" (1.5%); "there are several" (1.3%)

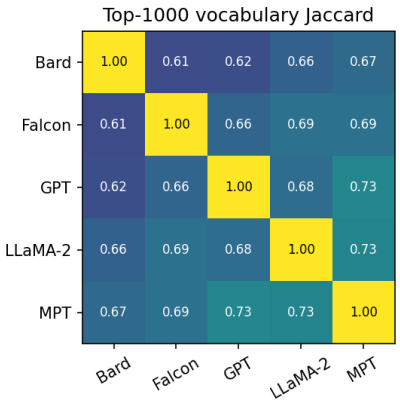




Vocabulary. We compared the 1,000 most frequent lower-cased words of each family on the training split. Overlap (Jaccard similarity) ranges from Bard–Falcon (0.61) to GPT–MPT (0.73). The vocabularies differ mostly in topic words and in "assistant" phrases, not in core vocabulary.

A classifier on hand-crafted features only. We built 34 features in five groups: length, lexical, markdown, punctuation and opening phrase (`src/features.py`). Logistic regression on these features reaches 55.2% test accuracy, and histogram gradient boosting reaches **68.2%** (macro-F1 68.3%). That is already better than our BiLSTM, using no word identities at all. The top permutation importances on the test set are `frac_blank_lines` (11.8 pts), `frac_first_person` (5.3 pts), `n_lines` (5.1 pts), `exclam_pw` (3.5 pts), `n_words` (2.9 pts).





Retraining gradient boosting with one feature group removed, or with only one group:

feature group	# features	acc. with this group only	acc. without this group	drop vs all features (pts)
length	4	41.5	66.4	-1.9
lexical	7	47.0	65.4	-2.9
markdown	7	47.9	61.9	-6.3
punctuation	9	46.3	63.7	-4.6
opening	7	36.3	66.6	-1.7

Ablations on the neural models. We re-train both neural models (3 seeds each) on transformed responses and compare with the full-text models:

model	ablation	test acc (%)	Δ vs full (pts)
CNN	none (full text)	65.6 ± 0.5	-
CNN	remove length (fixed 64-token window, short texts tiled)	58.9 ± 1.6	-6.8
CNN	strip markdown + line breaks	59.7 ± 0.7	-5.9
CNN	lowercase + strip punctuation	57.6 ± 2.3	-8.0
BiLSTM	none (full text)	59.6 ± 1.7	-
BiLSTM	remove length (fixed 64-token window, short texts tiled)	55.8 ± 0.4	-3.7

model	ablation	test acc (%)	Δ vs full (pts)
BiLSTM	strip markdown + line breaks	56.8 ± 0.7	-2.8
BiLSTM	lowercase + strip punctuation	56.1 ± 0.7	-3.5

(i) What signals do the classifiers use? The descriptive statistics show clear, human-readable differences:

- **Bard** writes the longest answers and is the heaviest markdown user: bullets in 37% and **bold** in 21% of responses. It typically opens with "Sure, here is…".
- **GPT** prefers numbered lists (42%), uses the most commas and the longest words; its most frequent opening, "As an AI…", appears in only 2.2% of answers, so GPT has no single signature opener.
- **LLaMA-2** is the most conversational: 31% of answers open with "Sure"/"Of course", 17% with "Hello", and it uses the most exclamation marks and first-person pronouns.
- **Falcon** writes the shortest answers (median 64 words) with almost no markdown, and is the family most likely to mention being an "AI language model" (10%).
- **MPT** sits in the middle on almost every statistic, which matches it being the hardest family to classify.

(ii) Linguistic, structural, or both? Both, with structure slightly ahead. With hand-crafted features only, the markdown group alone is the strongest single group (47.9%), and removing it hurts most (-6.3 pts). Lexical and punctuation features are close behind (47.0% and 46.3% alone), so no single group explains the families. The single most important feature is the fraction of blank lines, a pure layout signal. That features with no word identities reach 68.2%, above our BiLSTM, shows how much of the "fingerprint" is style rather than content.

(iii) Does removing signals hurt the neural models? Yes. Every ablation costs the CNN between 5.9 and 8.0 points:

- *Stripping markdown and line breaks:* -5.9 pts.
- *Lowercasing and removing punctuation:* -8.0 pts, the largest drop. This transformation also collapses line breaks, so it removes formatting too.
- *Removing length:* -6.8 pts. Caveat: this ablation also cuts the input from 256 to 64 tokens, so it mixes "no length information" with "less text".

The BiLSTM loses less (-2.8, -3.5, -3.7 pts), consistent with it exploiting surface formatting less effectively in the first place. Even after these surface cues are removed, both models stay far above chance (56–60%). The remaining signal is lexical: word choice, stock phrases and register.

5. Discussion and lessons learned

What worked.

- *Simple models are strong here.* A CNN with three kernel sizes is better than a BiLSTM (65.6 ± 0.5% vs 59.6 ± 1.7%) and 2.5× faster on CPU. Local n-gram patterns such as openings, list markers and stock phrases are exactly what convolutions with max-pooling detect, while the BiLSTM must carry them through a long sequence.
- *Baselines keep you honest.* TF-IDF + logistic regression (73.6%) beats both neural models. It sees the *whole* response, whereas our models see only the first 256 tokens, and the median Bard or GPT response is close to or above that length once punctuation tokens are counted. It also does not have to learn embeddings from only 16k training examples. A pretrained encoder, longer inputs or more epochs would be the obvious next steps; both neural models were still improving when they hit the 6-epoch cap.
- *Grouped splitting and leakage checks matter.* Splitting by prompt, and checking input-only accuracy against a source-only classifier, let us state with evidence that our RQ2 result is not inflated by prompt leakage.

What went wrong and what we would do differently.

- *Compute planning.* Our first BiLSTM took more than 10 min per epoch on a laptop CPU. We benchmarked and found sequence packing was 3× slower than masked pooling, and halved the hidden size. Profiling before launching a grid would have saved an hour.

- *Process hygiene.* Killing the Python workers but not the job queue left stale runs that silently overwrote four result files. We caught it because every result JSON stores its full configuration, and we validate those configurations before aggregating. We recommend this practice.
- *Cluster use.* Our first GPU jobs on the university cluster (ICDS Roar Collab) crashed. Falling back to CPU array jobs, one run per array task, finished the 42 RQ3/RQ4 runs in about 20 minutes, versus an estimated 2 hours on the laptop. Making every run resumable (skip if the result exists) made the mixed laptop/cluster workflow safe.

Limitations.

- *Dataset age and scope:* UltraFeedback responses come from 2023 models (GPT-3.5/4, Bard, LLaMA-2, Falcon, MPT). Today's Gemini, Claude, Qwen and DeepSeek are not covered, and fingerprints probably change with every model version.
- *System prompts:* UltraFeedback generated each response under one of several "principle" system prompts, which shift style for every model.
- *Family grouping:* merging gpt-3.5-turbo with gpt-4 and three LLaMA-2 sizes into one family hides within-family variation. Dropping the LLaMA fine-tunes (Vicuna, WizardLM, …) avoids, but does not resolve, the question of what a "family" is.
- *Balance:* classes are balanced by downsampling, but tasks and sources are not balanced within classes (e.g. Bard has no FLAN-NIv2 data). Task labels come from keyword rules and are noisy.
- *Statistics:* most configurations use 3 seeds, so the reported standard deviations are rough, and per-task test sets are small.
- *Hardware:* RQ1/RQ2 ran on a laptop CPU; RQ3/RQ4 ran on cluster CPUs, plus one GPU run. Two CNN runs repeated on the laptop reproduced their test accuracy exactly, but we did not check whether results are identical across different machines, and GPU kernels are not bit-deterministic.

References

- G. Cui et al. *UltraFeedback: Boosting Language Models with High-quality Feedback.* arXiv:2310.01377, 2023. Dataset: huggingface.co/datasets/openbmb/UltraFeedback (MIT licence).
- Y. Kim. *Convolutional Neural Networks for Sentence Classification.* EMNLP 2014.
- S. Hochreiter and J. Schmidhuber. *Long Short-Term Memory.* Neural Computation, 1997.